

METIN-LIKE MVP

Technical Spike & Tiny MMO Fork Plan

Godot 4 | multiplayer 3D | Metin 2 DNA | PoE-lite item economy

Decyzja: FORK Tiny MMO, ale traktujemy go jako donor infrastruktury, nie jako gotową bazę gameplayu.

Open-MT2 pozostaje referencją zachowania i domeny Metina (drop, inventory, questy, shop, protokół), nie serwerem docelowym.

[Tiny MMO repository](#) | [open-mt2 repository](#)

1. Decyzja architektoniczna

Tak: forkujemy Tiny MMO. Nie robimy jednak klasycznego forka typu „podmieniamy sprite rycerza na wojownika i jedziemy”. Tiny MMO ma wartościową warstwę sieciową i serwerową, ale aktualny gameplay jest mocno 2D/action-RPG. Nasz cel to 3D Metin-like, więc od pierwszego spike’a oddzielamy infrastrukturę od domeny gry.

```
Tiny MMO fork
|
+-- KEEP: gateway / master / world lifecycle
+-- KEEP: protocol / endpoints / wire codec
+-- KEEP: replication / AOI / instance management
+-- KEEP: auth / admin / basic persistence plumbing
|
+-- REPLACE: CharacterBody2D -> CharacterBody3D
+-- REPLACE: 2D movement/combat/AI
+-- REPLACE: inventory/item model
+-- REPLACE: quests/progression/content
+-- DELETE: demo assets and game-specific systems we do not want
```

Dlaczego fork, a nie kopiowanie kilku plików? Bo na początku jeszcze nie wiemy, które zależności Tiny MMO są naprawdę luźne. Fork zachowuje działający baseline, historię zmian i możliwość porównania z upstreamem. Dopiero po przejściu spike’a możemy „odchudzić” repo agresywniej.

2. Cel spike’a

Spike ma odpowiedzieć na jedno pytanie: czy infrastruktura Tiny MMO realnie skraca drogę do 3D Metin-like, czy 2D gameplay jest z nią tak sprzężony, że taniej będzie napisać cieńszą własną warstwę sieciową.

Minimalny pion techniczny:

```
Client A ----\
               > Gateway -> Master -> World
Client B ----/

1. login
2. wspólna mapa 3D
3. wzajemna replikacja ruchu
4. server-side movement validation
5. jeden mob
6. server-side combat
7. death + loot roll
8. ItemInstance w inventory
9. equip / upgrade / affix reroll
10. reconnect i persistence
```

3. Zakres spike'a - krok po kroku

3.1. 3D multiplayer

Tworzymy pusty World3D i MetinPlayer : CharacterBody3D. Sieć publikuje entity_id, position, rotation, velocity i animation_state. Klient może predykować ruch, ale serwer utrzymuje stan kanoniczny i waliduje max_speed, delta distance, map bounds i teleport threshold.

3.2. Pierwszy mob

Jeden WildDog z prostym stanem IDLE -> AGGRO -> CHASE -> ATTACK -> RETURN. AI wykonuje się wyłącznie na world serverze. Model danych rozdzielamy na MobDefinition (statyczny content) i MobEntity (runtime state).

3.3. Combat

Klient wysyła AttackRequest(target_entity_id). Serwer sprawdza zasięg, cooldown, stan atakującego i celu, a dopiero potem liczy damage. Klient odpowiada za animację/VFX, nie za wynik walki.

3.4. Item model

Od początku używamy ItemDefinition + ItemInstance. To konieczne dla +0..+9, unikalnych affixów, socketów, handlu i ekonomii. Nie kopiujemy prostego modelu Tiny MMO „item_id + amount” jako finalnego modelu equipmentu.

3.5. Loot

Na spike'u loot może trafiać automatycznie do inventory. Ważne jest server-side losowanie i utworzenie trwałego ItemInstance. Rendering itemu na ziemi nie jest ryzykiem architektonicznym, więc może poczekać.

3.6. Inventory/equip

Minimalnie: bag + weapon slot. EquipRequest idzie do serwera, serwer waliduje ownership/slot/requirements, przypina item UID do equipmentu i przelicza finalne staty.

3.7. PoE-lite crafting test

Dwie testowe waluty: Chaos Stone (reroll wszystkich losowych affixów) i Blessed Stone (dodaje jeden affix). To wcześniej weryfikuje, czy model ItemInstance jest wystarczająco elastyczny dla naszej ekonomii.

3.8. Persistence

Na spike'u SQLite jest wystarczające. Ważniejsze są osobne item_instances i inventory_slots niż trzymanie pełnego equipmentu jako jednego JSON blobu. Publiczny serwer można później przenieść na PostgreSQL.

4. Minimalny model domeny

ItemDefinition vs ItemInstance

ItemDefinition

- id
- type / subtype
- required_level
- base stats
- max_upgrade
- allowed_affix_groups

ItemInstance

- uid
- definition_id
- owner_id
- upgrade_level
- affixes[]
- sockets[]
- bind/trade flags
- created_at

AffixDefinition

AffixDefinition

- id
- group
- tier
- stat
- min_value
- max_value
- weight
- item_constraints

MVP nie potrzebuje setek affixów. Wystarczy kilka czytelnych grup: STR/DEX/VIT/INT, attack speed, crit, monster damage, boss damage, skill damage, life steal. Chodzi o ekonomię i decyzje, nie o skopiowanie złożoności PoE.

5. Tiny MMO - co zostawić, co wymienić, co wywalić

Najważniejsza zasada: nie zaczynamy od masowego kasowania. Najpierw fork musi odpalać się identycznie jak upstream. Potem robimy osobny commit „3D spike”. Dopiero po udanym połączeniu dwóch klientów kasujemy stare warstwy gameplayowe. Dzięki temu każdy etap ma działający punkt odniesienia.

Status	Obszar	Co robimy	Uwagi
KEEP	source/common/network	Wire protocol, endpoints, sync, codec - główny powód forka.	Nie modyfikować bez potrzeby na początku.
KEEP	source/server/gateway	Gateway/auth/routing.	Zostawić.
KEEP	source/server/master	Account/world orchestration + admin.	Zostawić, upraszczać dopiero po spike'u.
KEEP	source/server/world	World lifecycle, DB plumbing, server components.	Zostawić szkielet; gameplayowe komponenty oceniać osobno.
KEEP	addons/tinymmo	Plugin/tooling Tiny MMO.	Zostawić co najmniej do końca spike'a.
KEEP	addons/godot-sqlite	SQLite na prototyp.	Zostawić.
REPLACE	source/common/gameplay/characters	2D character hierarchy.	Nowy CharacterBody3D obok, potem usunąć 2D.
REPLACE	source/client/local_player	2D input/movement i local player.	Nie kasować przed uruchomieniem 3D replacement.
REPLACE	source/common/gameplay/combat	2D hitboxes/Area2D/action-RPG combat.	Można podpatrzeć reward/server authority, ale combat napisać pod Metina.
REPLACE	source/common/gameplay/items	Obecny item/inventory model.	Inspiracja pomocnicza; docelowo ItemDefinition + ItemInstance.
REPLACE	source/common/gameplay/maps	2D maps.	Nowe mapy 3D.
REPLACE	source/client/ui	UI Tiny MMO.	Zostawić tylko login/debug UI potrzebne do spike'a.
DELETE	assets/sprites	2D game sprites.	Po potwierdzeniu, że nie są wymagane przez ekran logowania/spike.
DELETE	assets/audio	Demo audio/content.	Niepotrzebne dla technicznego spike'a.
DELETE	source/common/gameplay/basing	System baz/territory Tiny MMO.	Nie jest częścią naszego MVP.
DELETE	source/common/gameplay/crafting	Crafting Tiny MMO.	Mamy własny PoE-lite model.

Status	Obszar	Co robimy	Uwagi
DELETE	source/common/ gameplay/mastery	Weapon mastery.	Poza scope MVP.
DELETE	source/common/ gameplay/sparring	Sparring-specific system.	Poza scope MVP.
DELETE	source/common/ gameplay/lighting	2D game-specific lighting.	Nowy 3D environment.
DELETE	source/common/ gameplay/weather	Game-specific weather.	Poza scope MVP.
DELETE	source/common/ gameplay/time	Day/time system.	Poza scope MVP.
DELETE	source/common/ gameplay/leaderboard	Leaderboards.	Niepotrzebne na spike/MVP core.
DELETE	source/common/ gameplay/jobs	Jobs system.	Nie pasuje do Metin MVP.

6. Linki - Tiny MMO prune map

- **KEEP - common/network:** [otwórz](#) - protocol, sync, endpoints, wire codec
- **KEEP - server/gateway:** [otwórz](#) - gateway, auth/routing
- **KEEP - server/master:** [otwórz](#) - accounts/world orchestration/admin
- **KEEP - server/world:** [otwórz](#) - world lifecycle, DB, server components
- **KEEP - addon tinymmo:** [otwórz](#) - plugin/tooling
- **KEEP - godot-sqlite:** [otwórz](#) - prototype persistence
- **REPLACE - characters:** [otwórz](#) - CharacterBody2D -> 3D
- **REPLACE - combat:** [otwórz](#) - 2D hitboxes/action-RPG combat
- **REPLACE - items:** [otwórz](#) - model itemów/inventory
- **REPLACE - local_player:** [otwórz](#) - 2D local movement/input
- **REPLACE - maps:** [otwórz](#) - 2D maps
- **REPLACE - client UI:** [otwórz](#) - docelowo własny UI
- **DELETE - sprites:** [otwórz](#) - demo 2D assets
- **DELETE - audio:** [otwórz](#) - demo audio/content
- **DELETE - basing:** [otwórz](#) - territory/base system
- **DELETE - crafting:** [otwórz](#) - Tiny crafting
- **DELETE - mastery:** [otwórz](#) - weapon mastery
- **DELETE - sparring:** [otwórz](#) - sparring
- **DELETE - lighting:** [otwórz](#) - 2D lighting
- **DELETE - weather:** [otwórz](#) - weather
- **DELETE - time:** [otwórz](#) - time/day-night
- **DELETE - leaderboard:** [otwórz](#) - leaderboards
- **DELETE - jobs:** [otwórz](#) - jobs

7. Co z systemami, które mogą się przydać później?

Nie kasowałbym od razu poniższych katalogów. Mogą zawierać gotowe wzorce serwerowe, które warto wykorzystać lub przepisać dopiero po uruchomieniu 3D:

- **source/common/gameplay/group** - party/group logic - może dać bazę pod party
- **source/common/gameplay/guilds** - guild service/persistence - potencjalnie przydatne po MVP
- **source/common/gameplay/trade** - trade flow - warto przejrzeć przed własnym handlem
- **source/common/gameplay/shops** - shop service - może pomóc przy NPC shops
- **source/common/gameplay/quests** - quest architecture - część idei może zostać
- **source/common/gameplay/dungeon** - instance/boss patterns - przydatne później
- **source/common/gameplay/events** - admin/live event patterns - opcjonalnie później

Te rzeczy klasyfikuję jako „QUARANTINE”: nie budujemy na nich MVP, ale nie wyrzucamy zanim nie przejrzymy zależności i wartości wzorca.

8. Proponowana kolejność commitów

1. Fork upstream i tag baseline (bez zmian).
2. Rename projektu, namespace/config, zachować LICENSE MIT.
3. Dodać branch spike/3d-network.
4. Dodać World3D + MetinPlayer3D obok istniejącego gameplayu.
5. Doprowadzić do A widzi B / B widzi A i server validation.
6. Dodać jeden server-side mob i prosty combat.
7. Dodać ItemDefinition + ItemInstance + persistence.
8. Dodać testowy loot/equip/+1/Chaos Stone.
9. Dopiero teraz usunąć 2D gameplay i demo content oznaczony DELETE.
10. Po cleanupie zrobić tag spike-success; od tego miejsca zaczyna się właściwe MVP.

9. Acceptance criteria

- [] Tiny networking działa z Node3D/CharacterBody3D.
- [] Dwa klienty stabilnie widzą wzajemny ruch.
- [] Serwer odrzuca oczywiste speed/teleport violations.
- [] Mob działa i podejmuje decyzje na serwerze.
- [] AttackRequest nie ufa damage z klienta.
- [] Death i respawn/despawn replikują się poprawnie.
- [] Serwer tworzy unikalny ItemInstance.
- [] Inventory persistuje przez logout/login.
- [] Equip zmienia finalne staty.
- [] Upgrade level zmienia staty.
- [] Chaos Stone zmienia affixy wyłącznie server-side.
- [] Reconnect nie duplikuje entity ani itemów.

10. Go / No-Go po spike'u

GO	Tiny MMO oszczędza nam networking, replication, auth i world lifecycle bez wymuszania 2D. Budujemy na forku.
----	--

NO-GO

Żeby uruchomić czyste 3D musimy przepisać replication/instance/network core. Wtedy zachowujemy wzorce Tiny MMO, ale budujemy cieńszą własną warstwę.

11. Referencje

- Tiny MMO - repo: [otwórz](#)
- Tiny MMO - MIT license: [otwórz](#)
- Tiny MMO - network layer: [otwórz](#)
- Tiny MMO - gameplay layer: [otwórz](#)
- open-mt2 - repo: [otwórz](#)
- open-mt2 - quest docs: [otwórz](#)
- open-mt2 - DropManager: [otwórz](#)

Wniosek: fork Tiny MMO jest sensowny, ale sukces projektu zależy od szybkiego odcięcia 2D gameplayu od infrastruktury. Spike ma to rozstrzygnąć zanim zaczniemy produkować content.